CUSTOMER FEATURE GUIDE

Zyvor QA Agent

Autonomous AI testing agent that reads requirements, generates tests, and validates every deploy.

An LLM-driven QA agent that continuously validates your web platform end-to-end — from requirement to running test to actionable report.

by ZyvorAI Labs

WELCOME

What is Zyvor QA Agent?

Zyvor QA Agent reads requirements from GitHub, generates Playwright tests, runs them after every deployment, detects regressions, and files plain-English reports — all coordinated by a LangGraph state machine. Beyond the pipeline it ships Mission Control, a live web console that runs 20+ QA capabilities on demand: end-to-end journey tests, API-contract and auth checks, real-time stream assertions, Core Web Vitals, visual sweeps, security probes, and recurring monitors. It is LLM-provider agnostic and degrades gracefully to rule-based fallbacks when no key is configured.

20+	5	3	10
QA actions in Mission Control	LLM providers supported	browser engines (Chromium/Firefox/WebKit)	network & security probes

What's inside this guide

GETTING STARTED

Access & first workflows

How to reach Zyvor QA Agent and get your first result. Assumes it has been deployed for you.

How to access it

WEB	Mission Control dashboard, served by `zyvor-qa serve` (e.g. `zyvor-qa serve --port 8080`, then open `http://localhost:8080/dashboard`). A live, self-refreshing console with a status hero, Actions panel (20+ QA capabilities as cards + a ⌘K command palette), Schedules, QA-run history, and Kubernetes pod health. Serve over HTTPS with `zyvor-qa serve --tls` (self-signed cert under `~/.zyvor-qa/tls`).
CLI	Primary interface is the `zyvor-qa` CLI (installed by `make install`). Verbs: `run`, `test`, `generate`, `discover`, `create`, `regression`, `serve`, plus the Mission Control actions `flow`, `route-sweep`, `api-test`, `auth-test`, `realtime`, `vitals`, `ai-test`. Examples: `zyvor-qa test` (smoke, no LLM), `zyvor-qa run --source github --spec docs/specs/my-feature.md`, `zyvor-qa generate --spec my-specs/products-page.md`, `zyvor-qa create "Verify the homepage Schedule Demo button is visible" --execute`.
API	The dashboard is a thin client over documented JSON endpoints you can script directly: `POST /api/dashboard/jobs {kind, params}` to run any action, `GET /api/dashboard/jobs/status` and `POST /jobs/cancel` to watch/stop, `GET /api/dashboard/overview` · `/pods` · `/events` · `/tests` · `/runs`, `GET/POST/DELETE /api/dashboard/schedules[/{id}]`, `GET /api/dashboard/jobs/report.{csv,html,pdf}`, and the HMAC-verified `POST /webhook/github`. `/health` and `/webhook/github` stay open even when login is on.
LOGIN	No auth by default (local-dev mode). Set `DASHBOARD_PASSWORD` (and optionally `DASHBOARD_USER`, default `admin`) to gate `/dashboard`, its API, and all artifacts behind the login screen; sessions are signed cookies (12 h, or 30 days with "remember me") and login is rate-limited (8 failures / 5 min per IP → 5-min lockout). `deploy-remote.sh` generates a per-host password automatically (skip with `--no-auth`). GitHub webhook payloads are authenticated separately by `GITHUB_WEBHOOK_SECRET` (HMAC).
NEEDS	Python 3.9+ (3.11 recommended) and Node.js 20+; copy `.env.example` to `.env` (at minimum set `ZYVOR_BASE_URL`); for GitHub sources run `gh auth login` (or set `GITHUB_TOKEN`) and `ZYVOR_PRODUCT_REPO=owner/repo`; an LLM key (`LLM_PROVIDER` + matching key) is optional except for `zyvor-qa create`.

Your first workflows

First run — smoke a site with no LLM or GitHub

1. Install: `cp .env.example .env` then `make install` (installs the `zyvor-qa` CLI, Playwright, and Chromium).
2. Set the target in `.env`: `ZYVOR_BASE_URL=https://zyvor.dev` (leave LLM and GitHub sections empty).
3. Run the hand-written smoke suite: `zyvor-qa test` — expect `Results: 11 passed, 0 failed`.
4. Run the full pipeline against the example spec: `zyvor-qa run --source local` (parse → generate → execute → report).
5. Read the report: `open reports/qa-summary.html` (or `npm run report` for the interactive Playwright report).

From a markdown spec to a running test

1. Write a user-story spec with an `## Acceptance Criteria` section, e.g. `my-specs/products-page.md`.
2. Generate without running: `zyvor-qa generate --spec my-specs/products-page.md`.
3. Inspect what the parser understood: `cat tests/fixtures/requirements.json`; rephrase any criterion that didn't become a step.
4. Run the full pipeline: `zyvor-qa run --source local --spec my-specs/products-page.md` (exit code is non-zero on failure, so it's CI-safe).
5. Optional: set `LLM_PROVIDER` + key in `.env` and regenerate for free-form prose and richer TypeScript (a quality gate falls back to the template if the LLM output is bad).

Add a one-off test from plain English

1. Configure an LLM in `.env` (this is the one feature with no non-LLM fallback): `LLM_PROVIDER=anthropic` + `ANTHROPIC_API_KEY=…` (or `ollama` for a local, free model).
2. Create and run in one step: `zyvor-qa create "Check that /vm page loads and mentions KubeVirt migration" --execute`.
3. Name the route and exact visible text in the description — they become `page.goto` and `getByText` assertions.
4. To keep the check permanently, promote the generated file from `tests/generated/` into `tests/manual/`, or turn the description into a versioned markdown spec.

Wire GitHub as the requirement source and comment on PRs

1. Authenticate: `gh auth login` (or set `GITHUB_TOKEN` in `.env`); token needs `Contents: Read` (+ `Pull requests: Write` for comments).
2. Point at the product repo in `.env`: `ZYVOR_PRODUCT_REPO=owner/repo` (owner/repo, not a URL) and `ZYVOR_BASE_URL`.
3. Run from a single spec and post results: `zyvor-qa run --source github --spec docs/specs/my-feature.md --pr-number 42`.
4. Or run from everything (`qa`-labeled issues + `docs/specs/` + README/CHANGELOG): `zyvor-qa run --source github`.
5. For automatic runs, set `GITHUB_WEBHOOK_SECRET`, start `zyvor-qa serve --port 8080`, and add a repo webhook to `https://<host>:8080/webhook/github` for `push`, `pull_request`, and `repository_dispatch`.

Turn on self-healing autofix for a nightly run

1. In `.env` enable diagnosis and repair: `ENABLE_AUTOFIX=true`, `ENABLE_AUTOFIX_APPLY=true`, `AUTOFIX_MAX_RETRIES=2` (keep `ENABLE_LLM_ANALYSIS=true` for useful suggestions).
2. Run the pipeline with a failing test: `zyvor-qa run --source local` — the fail branch runs analyze → autofix → apply_autofix → re-execute until green or retries are exhausted.
3. Review the patch like any diff: `git diff tests/` — self-healing fixes selectors, not intent, so confirm it isn't masking a real regression.
4. In CI prefer suggestions-only mode (`ENABLE_AUTOFIX=true`, `ENABLE_AUTOFIX_APPLY=false`) with a human-reviewed commit.

Operate live from Mission Control

1. Start the console: `zyvor-qa serve --port 8080` and open `http://localhost:8080/dashboard` (add `--tls` for HTTPS beyond localhost).
2. Run any capability from a card or press ⌘K (Ctrl-K) for the command palette; watch per-test ✓/✕ chips stream live, with a ■ Stop button and CSV/HTML/PDF download row.
3. Generate run history for the trends sparkline: `zyvor-qa run --source local` (each run appends to `reports/history/`).
4. Turn any job into a recurring monitor from the Schedules panel (5 min – 6 h) — e.g. smoke every 15 min, TLS check daily.
5. Point at a cluster (in-cluster SA or local kubeconfig) to activate the Pods/Workloads panels; set `DASHBOARD_PASSWORD` before exposing it since it reads pod logs.

SECTION 1

Autonomous Pipeline

A LangGraph state machine turns requirements into running tests and back into reports — no human in the loop.

🧬

AI Test Generation

Reads product requirements and generates ready-to-run Playwright test specs that assert the described behavior.

Turns written requirements into working coverage without hand-authoring every test.

How · CLI: `zyvor-qa generate --spec <file.md>` (parse → generate into `tests/generated/`), or Mission Control's ⚙ Generate card. Rule-based without a key; set `LLM_PROVIDER` + an API key in `.env` for free-form prose and richer TypeScript.

📄

Spec-to-Test from GitHub

Pulls a markdown spec (or all specs and issues) straight from your product repo and produces tests for it.

Requirements and their tests stay in sync because both live in the same repo.

How · CLI: `zyvor-qa run --source github --spec docs/specs/my-feature.md` (needs `ZYVOR_PRODUCT_REPO` in `.env` + `gh auth login`); omit `--spec` to fetch all `qa`-labeled issues, `docs/specs/`, and README/CHANGELOG. Dashboard: the ▶ Full pipeline card with `--source github`.

💬

Natural-Language Test Creation

Describe a check in plain English with `zyvor-qa create` and get a generated Playwright test, optionally run on the spot.

Anyone can add a test without knowing Playwright or selectors.

How · CLI: `zyvor-qa create "<description>" [--execute]`, or Mission Control's ✨ Create from English card. Requires an LLM (`LLM_PROVIDER` + key, or a local `ollama` model) — this is the one feature with no rule-based fallback.

🤖

Autonomous AI Browser Agent

Give a goal like 'create an Ubuntu VM with 2GB RAM' and the agent drives the real browser step by step to accomplish it.

Tests intent, not scripts — the agent figures out the clicks itself.

How · CLI: `zyvor-qa ai-test <url> --goal "<goal>" [--session <file>.json --insecure]`, or the 🤖 AI test card (URL, goal, session, max-steps). LLM decider by default (`LLM_PROVIDER` + key); a heuristic decider runs standard forms without a key.

Self-Healing Autofix

After a failure the LLM proposes selector repairs, and can optionally patch the spec and re-run until it passes.

Brittle selectors heal themselves instead of paging an engineer.

How · Config in `.env` : `ENABLE_AUTOFIX=true` for suggestions, add `ENABLE_AUTOFIX_APPLY=true` + `AUTOFIX_MAX_RETRIES=2` to patch files and re-run. Runs on the fail branch of `zyvor-qa run` ; review with `git diff tests/` .

Coverage Discovery & Expansion

Scans repo code and docs for untested routes and pages, reports the gaps, and generates tests to close them.

Surfaces the pages nobody remembered to test and fills them in.

How · CLI: `zyvor-qa discover --source github` reports gaps; `zyvor-qa run --source github --expand-coverage` (or `zyvor-qa generate … --expand-coverage`) generates the missing `coverage-*.spec.ts` . Or set `ENABLE_COVERAGE_EXPANSION=true` in `.env` for webhook/default GitHub runs.

> No API key? Parsing, generation, analysis, and summaries all fall back to rule-based implementations — only natural-language creation strictly requires an LLM.

SECTION 2

Browser & Journey Testing

Drive real user journeys across browsers and devices, and catch visual regressions frame by frame.

🎬

E2E Flow Tests

Drives a multi-step journey — log in, navigate, fill a wizard, assert the outcome — as one continuous session recorded to a single video and Playwright trace.

Watch the whole user journey succeed or fail, then time-travel debug it.

How · CLI: `zyvor-qa flow <url> --steps <file> | --describe "<journey>" [--video --username --password --insecure --session <file> --no-trace]`, or Mission Control's 🎬 Flow test card. Steps stream live; a `journey.webm` video and `trace.zip` land in `reports/jobs/<ts>-flow/`.

💨

Smoke Tests

Runs the hand-written smoke suite against any target with a single command and no LLM required.

A fast, dependency-free health check for every deploy.

How · CLI: `zyvor-qa test` (runs everything in `tests/manual/` with Chromium), or the ▶ Smoke card in Mission Control. Targets `ZYVOR_BASE_URL`; no LLM key needed.

🗺️

Route Sweep

Screenshots every route at desktop and mobile, then pixel-diffs each shot against a saved baseline, masking dynamic content.

Catches unintended visual changes across the whole site at once.

How · CLI: `zyvor-qa route-sweep <url> --routes "/,/pricing" [--mobile --update-baselines --auto --insecure]`, or the 🗺️ Route sweep card. Baselines persist under `reports/artifacts/route-baselines/`; tune the pass threshold with `VISUAL_MAX_DIFF_RATIO`.

👁️

Visual Regression

Compares captured screenshots against committed baselines with a configurable pixel-diff threshold.

Fails the build when the UI shifts unexpectedly, not when you meant it to.

How · Config in `.env`: `ENABLE_REGRESSION=true` (+ `REGRESSION_THRESHOLD`) compares against `screenshots/baselines/`. CLI: `zyvor-qa regression [--update-baselines]`, or the 👁️ Visual regression card.

🌐

Test Any Site

Crawls every page of any URL, generates a check per page, and runs them all — login and self-signed TLS supported.

Point it at a URL and get instant coverage with zero setup.

How · Mission Control's 🌐 Test any site card (URL, optional login, self-signed TLS toggle). For a standalone crawl into the coverage inventory, set `ENABLE_LIVE_CRAWL=true` (+ `CRAWL_MAX_PAGES` / `CRAWL_MAX_DEPTH`) or run `npm run crawl`.

🔀

Visual Compare

Pixel-diffs two live URLs side by side, such as staging against production, and produces a diff image and percentage.

Prove a deploy changed nothing it wasn't supposed to.

How · Mission Control's 🔀 Compare card — enter the two URLs; entirely UI/API-driven (`POST /api/dashboard/jobs {kind: "compare"}`) with no extra environment variables. Produces a side-by-side + diff image + % in the result panel.

🎲

Flaky Detection

Runs a suite N times and ranks each test by its flake rate.

Finds the unreliable tests before they erode trust in the whole suite.

How · Mission Control's 🎲 Flaky check card — pick the run count N; UI/API-driven with no extra environment variables. Results feed the Test health panel's flaky badges.

🧭

Cross-Browser & Device

Runs on Chromium, Firefox, and WebKit with device profiles and 3G/offline network throttling.

Confirms the experience holds up beyond your own laptop's browser.

How · Per-journey CLI flags: `zyvor-qa flow <url> --browser firefox|webkit --device "Pixel 7" --throttle 3g|offline` (set `ZYVOR_BROWSER` / `ZYVOR_DEVICE` / `ZYVOR_THROTTLE`), or the 🎬 Flow card's dropdowns. For the full pipeline, set `ENABLE_MULTI_BROWSER=true` in `.env` (install browsers with `npx playwright install --with-deps`).

SECTION 3

API, Auth & Real-Time

Validation that goes past the page — into your REST contracts, sessions, and live streams.

🔌

API Contract Tests

Validates REST endpoints against their OpenAPI schema and runs ordered multi-step API workflows with bearer or API-key auth.

Catches contract drift and broken endpoints before the UI ever sees them.

How · CLI: `zyvor-qa api-test <base> --spec <openapi.json> [--token "$JWT" --include-writes]` for schema validation, or `--workflow <file>.json` for ordered create→poll→delete steps with `{{variable}}` interpolation. Dashboard: the 🔌 API contract card (base URL, spec URL or inline JSON, optional bearer token).

🔐

Auth & Session Tests

Logs in, saves a reusable session, and asserts logout, expiry, and negative-auth behavior.

Verifies access control works — and hands other tests a ready session to reuse.

How · CLI: `zyvor-qa auth-test <url> --api-login /api/v1/auth/login | --login-url /login --username <u> --password <p> [--protected /dashboard --logout-url …]` , or the 🔐 Auth & session card. A passed run saves the session to `reports/artifacts/auth/<host>.json` for reuse via flow/realtime `--session` .

📡

Live-Data Assertions

Confirms WebSocket and SSE streams are actually delivering messages, covering reconnect, bearer/subprotocol/ticket auth, and live-region updates.

Proves real-time dashboards are live, not just loading.

How · CLI: `zyvor-qa realtime <url> --ws /path | --sse /path [--expect-messages 3 --token "$JWT" --subprotocol-jwt --ticket-url /path --live-selector <sel> --session <file>]` , or the 📡 Live data card. Flags a hung page `slow` via the `ZYVOR_SLOW_MS` latency budget instead of a false pass.

> A saved auth session can be reused by flow and real-time tests — and any target-site password is redacted from job status, history, and the live panel.

SECTION 4

Performance & Web Quality

Grade speed, accessibility, SEO, and code coverage on a single pass.

📊

Core Web Vitals

Measures and grades LCP, CLS, INP, FCP, and TTFB with device and network-throttle profiles.

Know how fast real users perceive the page, with a letter grade per metric.

How · CLI: `zyvor-qa vitals <url> [--throttle 3g --device "iPhone 14"]`, or Mission Control's 📊 Web Vitals card with device and throttle dropdowns. Each metric is graded good / needs-improvement / poor against Google's thresholds.

🔬

Site Audit with A–F Grade

Runs per-page accessibility (axe-core), links, SEO, console errors, performance, and security-header checks into a pass/warn/fail matrix and overall grade.

One report card for the whole quality picture of any page.

How · Mission Control's 🔬 Site audit card (enter the page URL) — UI/API-driven (`POST /api/dashboard/jobs {kind: "audit"}`) with no extra environment variables. Returns a pass/warn/fail matrix ending in an A–F health grade.

⏱️

Load Test

Fires N requests at a chosen concurrency and reports p50/p95/p99 latency and requests per second.

A quick throughput and latency read without a separate load tool.

How · Mission Control's ⏱ Load test card — set N requests and concurrency C; UI/API-driven with no extra environment variables (in-pod runs are capped to avoid resource exhaustion). Reports p50/p95/p99 latency and req/s.

📈

V8 JS Coverage

Collects Chromium V8 JavaScript coverage per test run and aggregates it into the report as a percentage.

Shows how much of your shipped JavaScript your tests actually exercise.

How · Config in `.env` : `ENABLE_V8_COVERAGE=true`, then run any suite (`zyvor-qa test` / `zyvor-qa run`). Artifacts land in `reports/v8-coverage/` and the percentage is summarized in the HTML/PR report.

SECTION 5

Network & Security Probes

One-shot checks that inspect the wire, the headers, and the certificate.

🔒

TLS Certificate Check

Resolves DNS and reports the certificate issuer, expiry, protocol, and SANs.

Catches expiring or misconfigured certificates before browsers do.

How · Mission Control's 🔒 TLS check card (enter the host/URL) — UI/API-driven with no extra environment variables, and schedulable from the Schedules panel (e.g. TLS check daily).

🔓

Exposed-Path Probe

Probes for sensitive paths such as `/.env` and `/.git/config`, aware of SPA fallbacks.

Flags accidental secret exposure that a functional test would miss.

How · Mission Control's 🧰 Probes card → Exposed paths (enter the base URL) — one-shot, UI/API-driven with no extra environment variables. SPA-fallback aware so a catch-all route isn't a false positive.

🍪

Cookie & Header Inspection

Reports Secure/HttpOnly/SameSite cookie flags, full response headers, CORS policy, and compression.

Verifies the security and caching posture of every response.

How · Mission Control's 🧰 Probes card → Headers / Cookies / CORS / Compression sub-probes — each renders as a table with flagged issues; UI/API-driven with no extra environment variables.

🔗

Redirect & Routing Probes

Traces the full redirect chain, robots/sitemap presence, sitemap URLs, DNS records, and API status with JSON-path assertions.

Confirms crawlers, DNS, and routing behave exactly as intended.

How · Mission Control's 🧰 Probes card → Redirects / Robots-sitemap / Sitemap URLs / DNS / API check sub-probes (the API check supports a JSON-path assertion) — UI/API-driven with no extra environment variables and schedulable.

Ten probes ship in total — redirects, headers, cookies, robots/sitemap, exposed paths, API check, sitemap URLs, DNS, CORS, and compression.

SECTION 6

AI Analysis & Reporting

Every run ends in an explanation a human can act on and a bundle they can share.

🧠

LLM Failure Analysis

Reads traces and screenshots after a failure to propose a root cause and a fix.

Turns a red X into a starting point instead of a mystery.

How · Config in `.env` : `ENABLE_LLM_ANALYSIS=true` (default). Runs automatically on the fail branch of `zyvor-qa run` , building root cause, affected area, and flake assessment from error messages, console/network logs, and artifact paths; a stub fallback runs without a key.

📝

Plain-English Summaries

Generates a human-readable run summary suitable for posting as a PR comment.

Reviewers see what changed and what broke without reading raw logs.

How · Config in `.env` : `ENABLE_LLM_REPORT=true` (default). The summary appears in the report and, when you add `--pr-number 42` to a `zyvor-qa run --source github` , is posted as the PR comment; a stub fallback runs without a key.

📄

CSV / HTML / PDF Reports

Writes a downloadable report bundle for every executed job, with per-failure likely-cause hints.

Share results in the format each audience actually wants.

How · Every executed job writes a bundle to `reports/jobs/<ts>-<kind>/` ; download it from the dashboard result panel's Download CSV · HTML · PDF row or via `GET /api/dashboard/jobs/report.{csv,html,pdf}` . PDF rendering is toggled by `ENABLE_PDF_REPORT` in `.env` .

📹

Video, Trace & Screenshot Artifacts

Captures journey videos, Playwright traces, and per-step screenshots, persisted under `reports/` and downloadable in bulk.

See exactly what the agent saw when something went wrong.

How · On by flow `--video` (or `ZYVOR_VIDEO=on` in `.env`); the Playwright trace is on by default (`--no-trace` to skip). Browse the 🎬 videos panel and ⬇ `all videos (zip)` in Mission Control; open a `trace.zip` at trace.playwright.dev. All persist under `reports/` (PVC-backed on Kubernetes).

🩺

Test Health & Trends

Ranks worst-offender tests by fail count and flake rate and tracks a pass-rate sparkline over the last 30 runs.

Spot the tests and trends that need attention over time.

How · Mission Control's Test health panel (worst-offender ranking) and QA Runs pass-rate sparkline (last 30 runs); scriptable via `GET /api/dashboard/tests` and `GET /api/dashboard/runs?limit=` . Each pipeline run appends to `reports/history/` .

SECTION 7

Mission Control Dashboard

A live console that runs every capability on demand and watches your cluster while it does.

🚦

Live Status Console

A self-refreshing dashboard with a glanceable verdict, stat tiles, and streamed per-test pass/fail output for the running job.

One screen tells you whether everything is green right now.

How · Run `zyvor-qa serve` and open `/dashboard` — the status hero shows ALL SYSTEMS GO / DEGRADED / SYSTEMS DOWN / CLUSTER OFFLINE and auto-refreshes every 5 s (press `r` to refresh now). Scriptable via `GET /api/dashboard/overview`.

🎛️

On-Demand Actions Panel

Launch any of 20+ QA capabilities from a card or the ⌘K command palette, with a stop button and live log.

Run any check without touching a terminal.

How · Click any Actions card, or press ⌘K (Ctrl-K) for the command palette; one job runs at a time with a ■ Stop button and live log. Scriptable via `POST /api/dashboard/jobs {kind, params}`, `GET /jobs/status`, `POST /jobs/cancel`.

⏰

Recurring Schedules

Turns any job into a recurring monitor from 5 minutes to 6 hours, re-triggered by a background scheduler.

Smoke every 15 minutes, audit hourly, TLS daily — set and forget.

How · Add/remove schedules from the dashboard's Schedules panel or the ⌘K palette (interval 5 min – 6 h; a background thread re-triggers due jobs, single-flight). Scriptable via `GET/POST/DELETE /api/dashboard/schedules[/{id}]`.

☸️

Kubernetes Pod Health

Shows per-pod phase, restarts, events, CPU/memory, and live log tails, with a restart button and namespace events.

Watch the platform under test and the agent's own pods in one place.

How · Activates automatically when a cluster is reachable (in-cluster service account, else local kubeconfig). Scope it with `DASHBOARD_NAMESPACE` and `DASHBOARD_POD_SELECTOR` in `.env`. Scriptable via `GET /api/dashboard/pods`, `…/pods/{name}/logs`, `DELETE …/pods/{name}` (restart).

🔑

Authenticated & TLS-Ready

Optional password login (rate-limited, signed-cookie sessions) gates the dashboard, API, and artifacts; serve over HTTPS with a self-signed cert.

Safe to expose a console that can read pod logs.

How · Config in `.env`: set `DASHBOARD_PASSWORD` (+ optional `DASHBOARD_USER`) to enable login; `/health` and `/webhook/github` stay open. Serve HTTPS with `zyvor-qa serve --tls` (self-signed under `~/.zyvor-qa/tls`) or `--tls-cert` / `--tls-key`.

🔧

Scriptable JSON API

The whole console is a thin client over documented JSON endpoints for jobs, schedules, runs, pods, and reports.

Automate the same actions the UI performs from your own tooling.

How · Call the JSON endpoints directly, e.g. `POST /api/dashboard/jobs {kind, params}` to run an action, `GET /api/dashboard/runs` for history, `GET /api/dashboard/jobs/report.pdf` for the bundle; read endpoints degrade to `"available": false` when no cluster is reachable.

SECTION 8

Integrations & Delivery

Wires into GitHub, your chat tools, your LLM of choice, and your cluster.

GitHub Integration

Reads specs and issues, runs on deploy events via an HMAC-verified webhook, and posts result summaries back as PR comments.

QA rides along with your existing GitHub workflow.

How · Set `ZYVOR_PRODUCT_REPO` in `.env` + `gh auth login` (or `GITHUB_TOKEN`), then `zyvor-qa run --source github [--pr-number 42]`. For automatic runs, set `GITHUB_WEBHOOK_SECRET`, run `zyvor-qa serve`, and add a repo webhook to `/webhook/github` for `push` / `pull_request` / `repository_dispatch`.

Slack, Teams & Email Alerts

Sends block-formatted Slack messages, Teams adaptive cards, and HTML email with the PDF report attached.

The right people hear about failures where they already work.

How · Config in `.env`: set `SLACK_WEBHOOK_URL`, `TEAMS_WEBHOOK_URL`, and/or `SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD` + `NOTIFY_EMAIL_TO` (PDF attached when available). Alerts fire on pipeline runs.

Provider-Agnostic LLM

Works with OpenAI, Anthropic, Azure OpenAI, Google, or a local Ollama model through a single configuration switch.

Use the model and vendor you already trust — or none at all.

How · Config in `.env`: `LLM_PROVIDER=openai|anthropic|azure|google|ollama`, `LLM_MODEL=<name>`, and the matching key (`OPENAI_API_KEY` / `ANTHROPIC_API_KEY` / `AZURE_OPENAI_API_KEY` + `AZURE_OPENAI_ENDPOINT` / `GOOGLE_API_KEY` / `OLLAMA_BASE_URL`). Omit entirely to use rule-based fallbacks.

Kubernetes & Docker Deploy

Ships manifests for Deployment, Service, Ingress, CronJob, RBAC, and PVC, plus a container image and a one-command remote deploy script.

Run it as a scheduled in-cluster job or a standing service.

How · Apply the manifests under `kubernetes/` with `make k8s-validate` then `make k8s-apply`; port-forward the dashboard (`kubectl port-forward svc/zyvor-qa-webhook 8080:80`). Or deploy to a host with `scripts/deploy-remote.sh <host> <user> [--service --tls]`.

CI/CD Workflows

Includes GitHub Actions for lint and unit tests, nightly and PR smoke runs, and post-deploy validation via repository dispatch.

Coverage runs automatically on push, PR, schedule, and deploy.

How · Ships GitHub Actions in `.github/workflows/` (lint, unit, nightly/PR smoke, post-deploy). Trigger the post-deploy pipeline from your own pipeline with `gh api repos/$ZYVOR_PRODUCT_REPO/dispatches -f event_type=staging-deployed -F 'client_payload[pr_number]=42'`.

Rust Diff Accelerator

An optional `zyvor-diff` Rust binary replaces Pillow for faster screenshot comparison.

Speeds up visual diffing on large baseline sets.

How · Config in `.env`: `ENABLE_RUST_PROCESSOR=true`; build the binary with `make rust` (or point `ZYVOR_DIFF_BINARY` at an existing build). Used by visual regression and route-sweep diffing instead of Pillow.

WRAPPING UP

Support & next steps

Support & contact

Zyvor QA Agent is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

> **Good to know:** Many features are opt-in behind flags (regression, autofix, coverage expansion, V8 coverage, multi-browser, Rust diff) and are off by default. Without an LLM key the agent still runs but uses rule-based fallbacks for parsing, generation, analysis, and summaries; only `zyvor-qa create` strictly requires an LLM. API validation checks HTTP statuses and OpenAPI schemas rather than full business logic. The Mission Control dashboard reads pod logs, so it is intentionally not exposed through the ingress and should be protected with a password. Kubernetes panels require a reachable cluster; without one they show an offline state while everything else keeps working. Multi-browser and load testing are best-effort in-pod and capped to avoid resource exhaustion.

Zyvor QA Agent

An LLM-driven QA agent that continuously validates your web platform end-to-end — from requirement to running test to actionable report.

info@zyvor.dev · zyvor.dev